

An Empirical Study on Distilling ChatGPT for Advancing Code Intelligence Tasks

Kang Yang*, Xinjun Mao*, Shangwen Wang*, Tanghaoran Zhang*, Bo Lin*, Yanlin Wang[†],
Yihao Qin*, Zhang Zhang*, Xiaoguang Mao*

* National University of Defense Technology

{yangkang, xjmiao, wangshangwen13, zhangthr, linbo19, qinyihao, zhangzhang14, xgmiao}@nudt.edu.cn

[†] Sun Yat-sen University, wangylin36@mail.sysu.edu.cn

Abstract—Pre-trained code models have emerged as crucial tools in various code intelligence tasks. However, their effectiveness heavily depends on the quality of the pre-training dataset, particularly the human reference comments, which usually serve as a bridge between the programming language and natural language. One significant challenge is that such comments can become inconsistent with the corresponding code as the software evolves. This discrepancy can lead to suboptimal training of the models, decreasing their performances.

Large language models (LLMs) have recently demonstrated superior capabilities in generating high-quality code comments. In light of that, we try to tackle the quality issue of the dataset by harnessing the power of LLMs. Specifically, we raise the question: *Can we rebuild the pre-training dataset by substituting the original comments with LLM-generated ones for more effective pre-trained code models?*

To answer the question, we first conduct a comprehensive evaluation to compare ChatGPT-generated comments with human reference comments. As existing reference-based metrics treat the reference comments as gold standards, we introduce two auxiliary tasks as novel reference-free metrics to assess the quality of comments, i.e., code-comment inconsistency detection and code search. Experimental results show that ChatGPT-generated comments demonstrate superior semantic consistency with the code compared to human reference comments, indicating the potential of utilizing ChatGPT to enhance the quality of the pre-training dataset.

Based on this finding, we rebuilt the widely used dataset, CodeSearchNet, with ChatGPT-generated comments. Subsequent experiments involve re-pre-training the widely used model CodeT5 with our refined dataset. Evaluation results on four generation tasks and one understanding code intelligence tasks show that the model pre-trained by ChatGPT-enhanced data outperforms its counterpart (pre-trained by original human reference comments data) on code summarization, code generation, and code translation tasks. This research validates the feasibility of rebuilding the pre-training dataset by ChatGPT for advancing code intelligence tasks and advocates rethinking the reliance on human reference comments for code-related tasks.

Index Terms—Code Summarization, Large Language Model, Pre-training, Code Intelligence Tasks

I. INTRODUCTION

In the realm of code intelligence, pre-trained code models have significantly enhanced a spectrum of tasks such as code summarization [1]–[3], code generation [4]–[6], code search [7]–[9], clone detection [10], [11] and automated bug fixing [12]–[14]. Pre-trained source code models such as CodeBERT [15], GraphCodeBERT [16], CodeT5 [17], and

UniXcoder [18] and have achieved state-of-the-art (SOTA) results on various software engineering tasks. The comments of the corresponding code snippets serve as a crucial bridge between the programming language (PL) and natural language (NL), providing contextual understanding pivots that are vital for pre-training the above models. As such, the effectiveness of the pre-trained code models relies heavily on the quality of their pre-training datasets [8], which traditionally depend heavily on human reference comments.

However, this reliance on comments introduces a fundamental challenge: as software evolves, these comments often become outdated or mismatched with the code [19]–[21], leading to semantic inconsistencies between the code and comment. Previous research has highlighted such inconsistencies. Shi et al. [22] revealed that 41.9% of the code summarization dataset TLC [23] is noisy, with 22.8% of the comments not matching the corresponding code snippets. Consequently, inconsistent comments deteriorate the quality of the PL-NL dataset, which can degrade the training efficacy and performance of code models. Sun et al. [8] identified that more than one-third of the comments in CodeSearchNet (Java) [24] did not describe core functionalities, and the model trained with noisy data faces severe performance degradation [8].

Recently, large language models (LLMs) like ChatGPT have demonstrated superior generation capabilities [25], [26], especially in generating high-quality code comments [27]. Specifically, the results show that when given adequate examples, the effectiveness of LLMs would be significantly higher than traditional deep learning-based approaches. In light of this, we are motivated to utilize LLMs to address the limitations faced by pre-trained code models. This paper aims to answer the following question: *Can we rebuild the pre-training dataset by substituting the original comments with LLM-generated ones for more effective pre-trained code models?*

To that end, we first conduct a comprehensive evaluation to compare LLM-generated comments with human reference comments. We investigate two types of LLMs published by OpenAI in this evaluation, including gpt-3.5-turbo and text-davinci-003. Traditional evaluation metrics for code summarization are reference-based, focusing on measuring the similarity between predicted comments and reference comments. However, the underlying assumption of the reference-based

evaluation is that reference comments are gold standard superior to other baselines, whether n-gram overlap or semantic similarity measurements are utilized. A critical purpose of this work is to compare the quality of ChatGPT-generated comments with human-written reference comments; thus, a reference-free evaluation is required to enable the comparison.

In our study, we introduce two auxiliary tasks – code-comment inconsistency detection and semantic code search – to offer a more refined reference-free assessment of the semantic similarity between the code and its associated comments. The inconsistency detection task aims to identify inconsistency between comment and code, and the code search task assesses the ability to retrieve the correct code snippet using its comment as a query. Our intuition is that a higher-quality comment would exhibit better semantic consistency with the corresponding code so that (1) it is less likely to be detected as inconsistent by a well-trained classifier and (2) it is expected to facilitate accurate retrieval of the associated code from a database when used as a search query. Our experimental results show that *comments generated by ChatGPT preserve better semantic consistency with the code than human reference comments*.

This insight forms the basis to reconstruct the popularly utilized CodeSearchNet [24] dataset, substituting the human reference comments with ChatPT-generated ones. We further extend our research by pre-training the widely used model CodeT5 [17] with this updated dataset and evaluating its performance across five downstream code intelligence tasks. The results of these experiments are revealing. The model trained with the ChatGPT-enhanced dataset exhibit superior performance in code intelligence tasks like code summarization, code generation, and code translation compared to the counterpart model trained with original datasets containing human reference comments. These findings affirm the potential of LLMs in improving the quality of training data for code intelligence tasks and underscore the need to reevaluate the longstanding reliance on human reference comments.

In summary, this research makes the following contributions:

- **Innovative Evaluation Metrics:** To the best of our knowledge, we are the first to adopt code comment inconsistency detection and code search to automatically evaluate the quality of code comments, which offer a reference-free approach to evaluating the quality of comments generated by ChatGPT as well as written by human.
- **Empirical Validation of ChatGPT:** Our experimental results demonstrate that ChatGPT-generated comments achieve better semantic consistency with code than human reference comments. This finding validates the quality and reliability of ChatGPT-generated comments in the context of software engineering.
- **Exploration of ChatGPT in Dataset Construction:** Our work explores using ChatGPT to create high-quality code comment datasets, demonstrating its practicality and effectiveness in training source code models and advancing code intelligence tasks. In addition, we rebuilt

the large-scale CodeSearchNet dataset with +2M high-quality comments generated by ChatGPT.

II. PRELIMINARIES

This section reviews the NL-PL pre-trained models, as well as the evaluation of code summarization.

A. NL-PL Pre-trained Models

A standard NL-PL pre-trained model involves initially training a large-scale model using self-supervised objectives on extensive unlabelled datasets, followed by fine-tuning it for specific downstream applications, such as tasks related to code understanding and generation, using task-specific loss functions. This paradigm, first introduced in natural language processing (NLP) [28], [29], has been adapted for programming tasks. Many NL-PL pre-trained models [15]–[17] have been proposed with their typical architectures and pre-training tasks.

CuBERT [30] and CodeBERT [15] are the pioneer models, and the CodeBERT is the first large NL-PL pre-trained model for multiple programming languages. Following BERT [28] and RoBERTa [31], multi-layer bidirectional Transformer [32] are utilized as the model architecture. CodeBERT is pre-trained on a bimodal dataset CodeSearchNet [24], and two objectives, Masked Language Modeling (MLM) [28] and Replaced Token Detection (RTD) [33], are employed as pre-training tasks. GraphCodeBERT [16], sharing CodeBERT’s architecture, goes a step further by incorporating the structural aspects of code, specifically the data flow graph (DFG). While maintaining the MLM objective, GraphCodeBERT does away with the RTD objective. Instead, it introduces two DFG-related tasks: predicting data flow edges and aligning nodes. In four downstream tasks, GraphCodeBERT has been shown to surpass CodeBERT’s performance [16].

The encoder-only based models necessitate an additional decoder for generation tasks, which cannot leverage the advantages of the pre-training phase. On the contrary, GPT-C [34] and CodeGPT [35] are pre-trained by using unidirectional language modeling, which is good at auto-regressive tasks like code generation, but is sub-optimal for code understanding tasks.

Recent studies have focused on encoder-decoder models to accommodate understanding and generation tasks. For instance, PLBART [36] utilizes the BART architecture [37] and is pre-trained on both NL and PL with denoising objectives. CodeT5 [17], developed by Wang et al., modifies the T5 model [38] to incorporate essential token type information from identifiers, facilitating multi-task learning in downstream applications.

In contrast to the prevailing focus in current research on optimizing model architectures and designing domain-specific pre-training tasks, our study shifts the spotlight to another critical factor influencing model performance: the quality of the pre-training data. This work investigates how enhancements in the pre-training dataset quality can influence the performance of pre-trained code models on downstream code intelligence tasks.

B. Evaluation of Code Summarization

Quantitatively evaluating the quality of the generated summary is a non-trivial task. Typically, this evaluation involves comparing the model-generated summary with a reference summary, which is often the original comment provided by a developer. These reference summaries serve as a crucial benchmark against which the generated summaries are assessed. The degree of similarity between the predicted summary and the reference summary is then calculated to evaluate the extent to which the predicted summary aligns with the reference, thus serving as an indicator of the quality of the predicted summary. These metrics are called reference-based ones and are typically classified into two main categories: n-gram overlap and semantic similarity [39], [40] in research communities.

The dominant evaluation methods are traditional n-gram overlap-based metrics like BLEU [41], which were originally utilized in the machine translation community for measuring the predicted translations' similarity to reference translations. BLEU evaluates the similarity between the predicted and reference summaries by comparing n-grams within them. Commonly, n varies from 1 to 4, allowing for the calculation of a $BLEU_n$ score to gauge performance:

$$BLEU_n = \frac{\sum_{t_n} \min\{Cp(t_n), Cr(t_n)\}}{P(n)} \quad (1)$$

In this formula, $BLEU_n$ represents the BLEU score for n-grams, $Cp(t_n)$ is the count of n-grams in the predicted summary, $Cr(t_n)$ is the count of n-grams in the reference summary, and $P(n)$ is the total number of n-grams in the predicted summary.

A limitation of BLEU is its design as a corpus-level measure rather than a sentence-level one. A comprehensive review revealed that BLEU's correlation with human judgment is notable only at the corpus level [42]. However, programmers typically meet individual code summaries in practice. This disparity in BLEU's applicability at the sentence level has been a point of contention among scholars across various disciplines [43], [44]. Although ROUGE [45] and METEOR [46]) have been proposed to address specific complaints about BLEU, they are word/n-gram overlap-based metrics. Essentially, they still compute literally whether the same words/n-gram appear in the same order in both predictions and references.

Measuring semantic similarity, as an alternative to word overlap methods, assesses word similarity within an embedding space, providing "partial credit" for word matches, as termed by Weiting et al. [47]. Rather than binary scoring, semantic similarity based metrics apply a graded score that reflects the semantic proximity of words in a pre-trained embedding. The application of semantic similarity to code summary assessment is supported by growing evidence of the insufficiency of mere word overlap [48], [49]. Mahmud et al. advocate for BERTScore as it may more effectively capture semantic similarities than traditional symbolic similarities [49]. Haque et al. explore word-based and sentence-based semantic similarity metrics in assessing code comment summarization.

They conclude that the cosine similarity derived from the Sentence-BERT Encoder [50] and Universal Sentence Encoder [51] sentence representations most closely aligns with the similarity perceived by human evaluators.

Reference-based metrics, including n-gram overlap and semantic similarity based ones, are traditionally used on the assumption that human-written comments serve as a benchmark for quality. However, these metrics are primarily designed to assess the similarity between machine-generated comments and these human-written references rather than directly comparing the quality of one against the other. In the context of LLMs, which can generate comments with high levels of proficiency, these metrics may not fully capture the nuances of quality, especially when it comes to determining whether the machine-generated content is of equal or superior quality compared to the human references.

III. EVALUATION OF CHATGPT-GENERATED COMMENTS

A. Objective and Research Questions

This study investigates the potential of ChatGPT to rebuild training datasets for advancing code intelligence tasks. Before constructing the dataset, we focus on the comprehensive comparison of ChatGPT-generated comments and human reference comments in this section. Traditional evaluation approaches, such as n-gram overlap based metrics and semantic similarity based metrics, are reference-based evaluation approaches. However, these reference-based metrics cannot directly compare the quality of ChatGPT-generated comments and human reference comments because references are seen as gold standard. Recognizing this gap, our study proposes a paradigm shift in evaluation strategies. We focus on the practical utility and real-world applicability of the comments, moving beyond traditional metrics to extrinsic evaluation tasks. To address this, we utilize two code intelligence tasks: code-comment inconsistency detection and semantic code search, which effectively measure the semantic alignment between the code and its corresponding comments. Thus, these two tasks are utilized as reference-free metrics for extrinsic comment evaluation. Guided by the two principal research questions (RQs), our study explores:

RQ1: How effective are the proposed reference-free metrics in assessing the quality of code comments? This question explores the effectiveness of our newly proposed evaluation metrics and examines their alignment with traditional reference-based metrics, which lays the foundation for a comparative evaluation of ChatGPT-generated comments and human reference comments.

RQ2: How does the quality of comments generated by ChatGPT compare to human-written comments? This question investigates the direct quality comparison of ChatGPT-generated comments and human reference comments, which is central to determining the practicality of employing ChatGPT to enhance training datasets for code intelligence tasks.

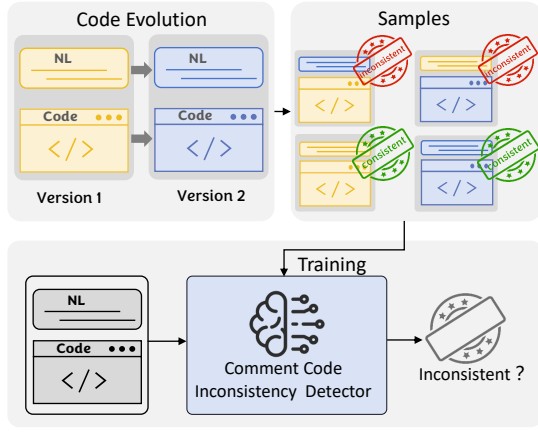


Fig. 1. Code comment inconsistency detection as the reference-free evaluation metric.

B. Reference-free Evaluation Metrics

1) *Code Comment Inconsistency Detection*: The task of code comment inconsistency detection (CCID) is to determine whether a comment is semantically misaligned with the corresponding code snippet [21]. Since CCID is of immense practical use to software developers who have a vested interest in keeping their code bases easily readable, navigable, and as bug-free as possible, prior works proposed kinds of approaches for detecting inconsistencies [52], [53].

Our motivation stems from the hypothesis that the most informative comments are those that accurately mirror their corresponding code. Inconsistent comments can cause confusion, errors, or misinterpretations, reducing effectiveness. The CCID task is designed to assess the consistency between comments and their code quantitatively. This enables us to identify discrepancies that may indicate outdated or erroneous comments, providing a precise and objective means to evaluate comment quality. Furthermore, the ability to detect inconsistency is intrinsically linked to a deeper understanding of the code-comment relationship and can serve as a valuable tool for software maintenance. By focusing on the alignment of code and comments, we can offer insights into the utility of comments and their role in enhancing the clarity and correctness of software documentation. Therefore, we introduce the CCID task to evaluate the quality of code comments by measuring the inconsistency rate in test datasets.

The data we used to train the CCID classifier is curated by Panthaplackel et al. [52]¹, which includes 40,688 samples of @return, @param, and summary Javadoc comments paired with their corresponding Java code methods. They consider comment-code pairs from each version of consecutive commits: $(c_1, nl_1), (c_2, nl_2)$. We collect examples that code changes do exist between two versions in which $c_1 \neq c_2$. As a result of code changes, the developer updated the comment because it would have otherwise become inconsistent. There-

fore, if $nl_1 \neq nl_2$, we take nl_1 comment to be inconsistent with c_2 code. As illustrated in Figure 1, (c_1, nl_2) and (c_2, nl_1) are consequently constitute the inconsistent positive examples. In contrast, if $nl_1 = nl_2$, the collected examples (i.e. (c_1, nl_1) and (c_2, nl_2)) are labelled as consistent negative examples. The assumption behind this is that the developer chose not to update the comment while modifying the code, as the comment was still consistent with the changes [52]. Figure 1 demonstrates the data constructing and training process of the CCID classifier.

Nevertheless, this procedure of collecting data brings in inaccuracies [52]. If a comment is refined without changing its semantics, there are positive mislabelled examples. Hence, even though $nl_1 \neq nl_2$ and $c_1 = c_2$, the comment is actually consistent with code. To alleviate this noise, we empirically consider the $nl_1 \neq nl_2$ examples that there is a difference of more than 30% in tokens between them in our work. The assumption is that 30% tokens' difference largely results in semantic changes in comments evolution.

Suppose f denotes the CCID classifier, and the input of f is a code snippet c and its corresponding comment nl . The output $f(c, nl) = 1$ indicates they are semantically inconsistent; otherwise, $f(c, nl) = 0$. Assume $\langle C, NL \rangle$ is the code snippets, and its paired model-generated/human-written comments in test datasets, and the total examples in test datasets is N . We define the inconsistency rate (*IncRate*) by calculating the proportion of inconsistent examples.

$$IncRate = \frac{1}{N} \sum_{i=0}^N f(c_i, nl_i) \quad (2)$$

We propose to use *IncRate* as a reference-free metric to evaluate the quality of comments, and the lower *IncRate* indicates better semantic consistency between the code and the comments.

2) *Semantic Code Search*: The semantic code search task is to retrieve a code snippet that matches a given query by effectively capturing the semantic similarity between the query and code. Semantic code search is a vital software development assistant that significantly improves development efficiency and quality.

Our motivation is rooted in the premise that the code search task could serve as an indicator of the degree to which comments are aligned with their corresponding code. We assume that high-quality comments should not only elucidate code functionality but also enhance the discoverability of code snippets through semantic code search. This assumption aligns with the practical use case of developers who regularly rely on comments to navigate and understand large code bases. Additionally, code comments are used as an alternative to the practical queries in research communities [8], [24]. Consequently, the effectiveness of comments in facilitating accurate and efficient code search results serves as a proxy for their quality, providing a concrete, measurable dimension to an otherwise subjective attribute of software documentation. Therefore, we argue comments that are more helpful for code

¹<https://drive.google.com/drive/folders/1heqEQGZHgO6gZzCjuQD1EyYertN4SAYZ>

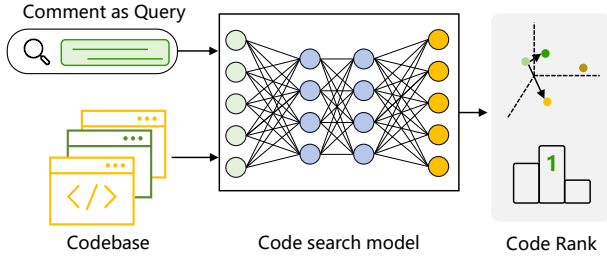


Fig. 2. Code search as reference-free evaluation metric.

search tasks are likely to be of higher quality. In our study, we employ comments as queries within a code search task to assess the comment quality.

To evaluate the performance of code search, we use the Mean of Reciprocal Rank (MRR) [7], [54], [55], which has been widely adopted in the evaluation of semantic code search. The MRR score quantifies the ranking of the target code snippet to the given comment query, and it only cares about where the most relevant result is ranked. We use CodeBERT [15] for code search task, and follow the configuration reported in their artifacts² to construct data and fine-tune model. When computing MRR scores in the testing set, for each query-code pair, $|Q| - 1$ code snippets from other pairs in the same batch play the role of distractor codes, where $|Q|$ is the batch size, and we set as 1000 in this work. For a single batch, MRR is calculated as follows:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{Rank(\tilde{c}_i, nl_i)} \quad (3)$$

where $\tilde{c}_i$ is the ground-truth code snippet for its paired comment query nl_i , and $Rank(\tilde{c}_i, nl_i)$ is its corresponding rank in the retrieved results. MRR gives a score of the predicted result based on its rank. The average value of all batches is the final MRR score.

We propose to use MRR as a reference-free metric to evaluate the quality of comments. A higher MRR indicates better semantic consistency between the code and the comments.

C. Experiment Design

1) *Dataset*: We conduct experiments on a widely used Java benchmark dataset TLC [23], which has 66k code-comment pairs collected from more than 9K open-source Java projects created from 2015 to 2016 with at least 20 stars. They first extracted Java methods and Javadocs and treated the first sentence of the Javadoc as the ground-truth comment of the corresponding code. We directly use the TLC dataset open-sourced by the previous work [22]. The training/validation/test set contains 53,528/7,555/4,985 samples, respectively.

2) *Baselines and ChatGPT*: In our research, we aim to conduct a comprehensive analysis of comment quality, comparing the performance of ChatGPT with that of human-written comments. To provide a meaningful context for evaluation,

we introduce several deep learning (DL) based code summarization models as baselines. Specifically, we incorporate NCS [56], SIT [57], and DOME [58] as benchmarks to establish a foundation for assessing the quality of ChatGPT-generated comments and human reference comments. This comprehensive comparison will facilitate a more nuanced understanding of the performance differences in DL model-generated, ChatGPT-generated, and human-written comments, thereby enhancing the overall insight of our analysis.

NCS [56] is a transformer-based approach that uses relative distances instead of absolute positions in the self-attention computation and applies a copy mechanism to copy rare tokens from the input source code.

SIT [57] is a structure-induced Transformer that integrates ASTs structure features into the self-attention calculation, which combines the AST tree, data-flow graph, and control-flow graph as a multi-view graph matrix to filter attention connections between tokens.

DOME [58] is an approach that takes the comment intent information into account, which utilizes the intent-guided selective attention to explicitly select intent-relevant information from the source code, and produces various comments reflecting different intents. We use the same data split and follow the default training parameters reported in these papers.

ChatGPT³ has become one of the most attention-grabbing achievements in the era of AI-generated content (AIGC). ChatGPT also has attracted widespread attention from software engineering as it exhibits excellent capability in software development. In this work, we investigate their performance in comment generation task, and two kinds of models published by OpenAI are taken into account. These two models are powerful enough and easy to access, leaving the time long enough for reproducing before it is deprecated.

- **gpt-3.5-turbo**. The most capable and cost-effective model in the GPT-3.5 family, which can understand and generate natural language and code. The snapshot of gpt-3.5-turbo from June 13th, 2023, gpt-3.5-turbo-0613⁴, is the version used in this work.
- **text-davinci-003**. The text-davinci-003 is the reinforcement learning with reward model trained from comparisons by humans, which is 10 times more expensive than gpt-3.5-turbo.

To obtain summaries written by gpt-3.5-turbo/text-davinci-003, we design a simple prompt with the following format shown in Figure 3.

We collected summaries from gpt-3.5-turbo/text-davinci-003 using this prompt for a total of 66k Java methods, including 53,528 from the training set, 7,555 from the validation set, and 4,985 from the test set, respectively. As ChatGPT tends to give much longer comments than human-written references, we set the parameter *max_tokens*=20 to meet the average length of human references when requesting OpenAI API for a fair comparison. Table I displays the statistics of word

²<https://github.com/microsoft/CodeBERT/tree/master/CodeBERT/codeSearch>

³<https://openai.com/chatgpt>

⁴<https://platform.openai.com/docs/models/gpt-3-5>

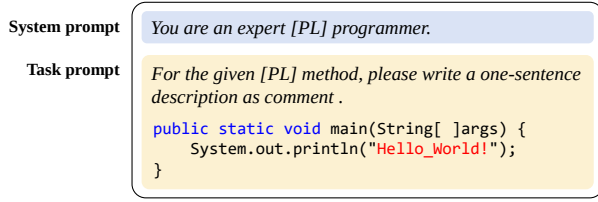


Fig. 3. The prompt design for code comment generation.

TABLE I
STATISTICS OF THE AVERAGE LENGTH OF WORDS AND THE UNIQUE WORDS APPEARED IN HUMAN REFERENCE COMMENTS AND CHATPT-GENERATED COMMENTS ON TLC DATASET

TLC dataset	Avg.words length	Unique words
human references	11.78	32,909
gpt-3.5-turbo	13.38	40,619
text-davinci-003	11.94	36,829

length and unique words for ChatGPT generated and reference comments. As for the sampling temperature parameters, we set the $top_p = 1$ and the $temperature = 1$.

3) *Evaluation Metrics*: It is also necessary to report some commonly used traditional metrics in research communities for comparison as we propose new evaluation metrics. On one hand, this allows us to compare the consistency, strengths, and limitations of different metrics. On the other hand, it could offer effective evidence of our proposed methods when comparing the ChatGPT-generated comments with human reference comments. Therefore, we categorize the evaluation metrics into two main groups based on whether references are needed as a standard: (1) reference-free metrics and (2) reference-based metrics, as shown in Figure 4.

IncRate is a reference-free metric for code summarization. As introduced in the former subsection III-B1, IncRate evaluates the quality by measuring the percentage of inconsistent comment-code pair examples in the test set.

MRR is originally used for the evaluation of information retrieval. As detailed in former subsection III-B2, we adopt it to measure the semantic relationship between the comment query and its paired code snippet. Higher MRR means a better quality of comment in semantic coherence.

USE [40] is a metric that encodes the reference and the predicted summary to a fixed-length vector using a universal encoder and computes the similarity scores between two summaries. In this work, our experiments use the *pre-trained universal-sentence-encoder-large* model [59].

BLEU [41] is a textual similarity metric that calculates the precision of n-grams in a translated sentence compared to a reference sentence, with a weighted brevity penalty to punish short translations. We use the standard BLEU score which provides a cumulative score of uni-, bi-, tri-, and quat-grams.

ROUGE [45] is a popular automatic evaluation metric that is recall-oriented. It computes the count of several overlapping units such as n-grams, word pairs, and sequences. ROUGE has several different variants from which we consider the ROUGE-L.

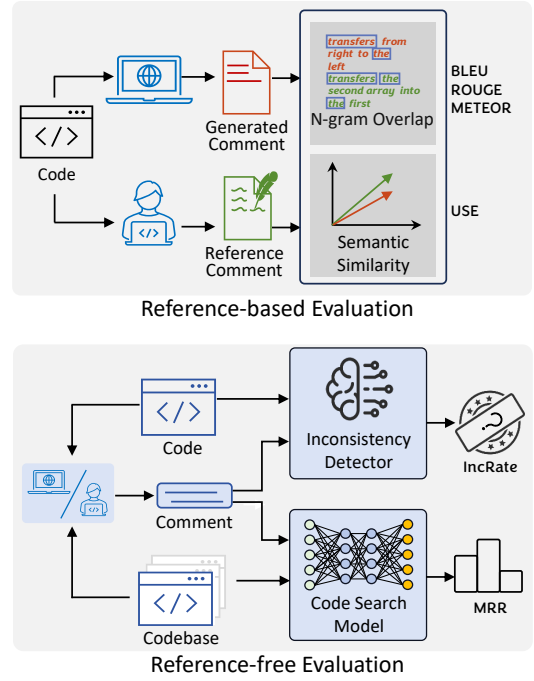


Fig. 4. Reference-based and reference-free evaluation.

METEOR [46] is a metric based on the general concept of unigram matching, and it combines precision, recall, and a custom score determining the degree to which words are ordered correctly in the translation.

D. Evaluation Results and Analysis

The semantic similarity based metric USE reveals a greater discrepancy among the NCS, DOME, and SIT models, highlighting the limitations of traditional n-gram overlap-based metrics like BLEU, ROUGE, and METEOR in capturing subtle semantic differences. This is evident from the initial three rows of Table II, where the performance of NCS, DOME, and SIT models appears closely matched, with no more than 2% difference in BLEU, ROUGE, and METEOR scores. Such marginal disparities challenge the intuitive differentiation of model efficacy. The traditional n-gram overlap-based metrics merely measure the literal proximity of predicted comments to reference comments, yet many words have close synonyms and certain words within a sentence carry more weight than others [40]. While comments generated by these three DL-based models may seem similar on a superficial literal level, the USE metric, which assesses semantic similarity through embedding vectors, provides a nuanced ability to distinguish the quality differences between the models' generated comments.

Our proposed reference-free metrics, IncRate and MRR scores, align with the four traditional reference-based metrics in evaluating three DL-based models. It is observed that SIT generated comments exhibit the lowest IncRate, DOME performs the second and NCS generated comments get the highest inconsistency rate with their corresponding code snippet.

TABLE II
EVALUATION RESULTS OF CHATPT AND THREE DL-BASED MODELS

Code Sum Methods	reference-based				reference-free	
	BLEU	ROUGE	METEOR	USE	IncRate	MRR
NCS	21.55	35.98	15.08	0.5198	31.84%	0.5614
DOME	22.20	36.67	16.47	0.5460	24.99%	0.6296
SIT	22.54	37.87	16.11	0.5634	23.65%	0.6404
Human References	-	-	-	-	15.05%	0.8165
gpt-3.5-turbo	15.24	33.07	16.17	0.6164	5.33%	0.8733
text-davinci-003	17.49	34.69	15.38	0.6234	3.65%	0.8826

pets. Simultaneously, SIT generated comments also achieve the highest MRR scores when they are used as code search queries among three DL-based baselines, while DOME ranks second and CSN performs the lowest MRR score. Furthermore, our proposed two reference-free metrics can reveal more substantial performance differences among NCS, DOME, and SIT compared to reference-based metrics. While n-gram-based metrics show less than a 2% difference and USE metrics less than 5%, IncRate and MRR metrics demonstrate a wider gap of 8%. This highlights the effectiveness of IncRate and MRR in detecting subtle but crucial differences in comment quality across the DL-based code summarization models.

In comparing two ChatGPT models, gpt-3.5-turbo and text-davinci-003, with three DL-based baseline models, it's clear that ChatGPT falls behind in BLEU, ROUGE and METEOR scores but surpasses all three DL-baselines in the USE metric. This discrepancy indicates that relying solely on reference-based metrics could yield unreliable or contradictory assessments. Notably, as shown in Table I, two ChatGPT models exhibit a richer vocabulary in their comments, 12%-23% larger than human written reference comments. The DL-based comments share the same vocabulary with human references. This suggests that ChatGPT-generated comments preserve more semantic diversity in word/token selection. Therefore, the results underscore the importance of semantic measurement over literal overlap for a more comprehensive evaluation of ChatGPT's capabilities.

Based on the semantic similarity metric USE, ChatGPT-generated comments show better likeness with human reference comments than three DL-based baselines. According to our reference-free metrics, both two ChatGPT models achieve lower IncRate and higher MMR scores than all three DL-based baselines, which aligns well with the conclusion obtained by the USE metric.

Answer to RQ1 The reference-free metrics, IncRate and MRR, are effective in assessing code comment quality as they not only align with reference-based metrics within homogeneous models but also reveal more substantial performance differences across various models.

The USE is a reference-based metric, although it can measure the semantic similarity between predicted comments and human reference comments. An inherent flaw rooted in

the reference-based metric is that since human references are treated as the gold standard, we cannot fairly compare the quality of human reference comments with model-predicted comments.

In contrast, IncRate and MRR excel as reference-free automatic evaluation metrics, which are independent from reference standards. Their focus on the semantic relationship between a comment and its corresponding code enables a direct assessment of comment quality. The IncRate and MRR provide objective approaches to evaluate comment quality, regardless of whether the comment is produced by summarization models, generated by ChatGPT, or written by human developers.

As shown in Table II, 15.05% of human reference comments in the TLC test set are detected as semantically inconsistent by the inconsistency detection classifier, suggesting that non-standard practices of co-evolving and maintaining comments with code often exist in software development and evolution. In comparison, the IncRate for gpt-3.5-turbo and text-davinci-003 generated comments are markedly lower, at 5.33% and 3.65%, respectively. Additionally, in the code search task where comments serve as queries, ChatGPT-generated comments also surpass human references. Specifically, gpt-3.5-turbo and text-davinci-003 achieve MRR scores of 0.8733 and 0.8826, outperforming the human reference score of 0.8165. The quality of comments generated by the two ChatGPT models, text-davinci-003 and gpt-3.5-turbo, shows a minor difference. The text-davinci-003 produces slightly superior comments, while it is ten times more expensive than gpt-3.5-turbo.

Answer to RQ2 The ChatGPT-generated comments possess higher quality than human reference comments by preserving lower inconsistency and higher semantic relevance to the corresponding codes.

IV. DISTILLING CHATGPT FOR CODE INTELLIGENCE TASKS

In the realm of pre-trained source code models, the quality of training data is fundamental to the model's performance. The PL-NL paired data is crucial for bridging the semantic gap between programming and natural language, providing context and descriptive insight that facilitate code-intelligence tasks. Section III reveals that ChatGPT-generated comments exhibit higher semantic relevance and lower inconsistency with the

TABLE III
STATISTICS OF CGPT-CSN DATASET

PLs	W/ ChatGPT-NL	W/o NL
Ruby	53,269	110,551
JavaScript	138,577	1,717,933
Go	346,333	379,103
Python	457,429	657,030
Java	496,651	1,070,271
PHP	578,072	398,058
Total	2,070,331	4,332,946

corresponding code snippets than human reference comments. These findings indicate that incorporating ChatGPT-generated comments could significantly enhance the training data quality, thereby motivating the exploration of rebuilding pre-training dataset for code intelligence tasks. Therefore, we ask the following research questions:

RQ3: How does the pre-training data rebuilt by ChatGPT impact the performance of the downstream code intelligence tasks?

A. Experiment Design

To assess the impact of ChatGPT-rebuilt data on code intelligence tasks, we conduct experiments as follows: First, we reconstruct a popularly used dataset by substituting human-written comments with ChatGPT-generated ones. This step ensures improved semantic consistency between the NL comments and PL code snippets within the dataset. Then, the rebuilt dataset is utilized to pre-train a widely used source code model. After that, fine-tune the pre-trained model in downstream tasks and measure their performances. We finally evaluate the impact of ChatGPT-rebuilt pre-training data by comparing the outcomes of the model trained on the rebuilt dataset with model trained on original human-referenced dataset, both quantitatively and qualitatively.

1) *Dataset and Pre-trained Model*: We rebuild the popular dataset CodeSearchNet [24] by replacing the human reference comments with ChatGPT generated ones, noted as cgpt-CSN. The prompt we used is shown in Figure 3. As the empirical finding in Section III, the quality of comments generated by text-davinci-003 and gpt-3.5-turbo shows a minor difference. The text-davinci-003 is slightly better than gpt-3.5-turbo while it is ten times more expensive than gpt-3.5-turbo. Due to considerations of OpenAI API cost and computation cost, we utilize gpt-3.5-turbo for rebuilding the dataset, and we did not incorporate the C/Sharp dataset added in the original CodeT5 [17]. We utilized approximately 2.07 million paired PL-NL instances, encompassing six PL: Java, Python, PHP, Javascript, Go, and Ruby. The statistics of the rebuilt CodeSearchNet cgpt-CSN are displayed in Table III.

In this study, we choose CodeT5 [17], a widely-used pre-trained model for source code based on an encoder-decoder framework similar to T5 [38], as our pre-training model for downstream code intelligence tasks. We follow Feng et al. [15] to employ CodeSearchNet [24] to pre-train CodeT5, which consists of six PLs with both unimodal and bimodal data. It is trained with four pre-training tasks, including Masked

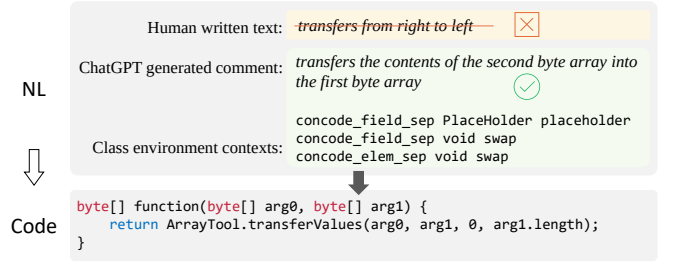


Fig. 5. A Concode case rebuilt by ChatGPT.

Span Prediction (MSP) task, Identifier Tagging (IT), Masked Identifier Prediction (MIP), and Bimodal Dual Generation (BDG).

As the pre-training implementation is not available, we re-implement the CodeT5 pre-training process based on Huggingface's T5 [38] PyTorch implementation, and the size of model is 220M, same as CodeT5-base. We set the maximum source and target sequence lengths to be 512. We use the mixed precision of FP16 to accelerate the pre-training. We set the batch size to 48 and employed the peak learning rate of $2e-5$ with linear decay. Following the settings in CodeT5 [17], we pre-train the model with the denoising objective for 100 epochs and bimodal dual training for further 50 epochs. In the fine-tuning phase, for the hyperparameters in the CodeXGLUE [35] tasks, such as learning rate, training steps, and batch size, we follow their default settings.

2) *Code Intelligence Tasks and Metrics*: For the code intelligence tasks, we cover 4 generation and 1 understanding tasks in the CodeXGLUE benchmark [35] and employ the provided public datasets and the same data splits following it for all these tasks. We first consider two cross-modal, text-to-code and code-to-text generation tasks, two code-to-code generation tasks and one code understanding task.

Code summarization aims to summarize a function-level code snippet into natural language descriptions. The dataset consists of six PLs including Ruby, JavaScript, Go, Python, Java, and PHP from CodeSearchNet [24]. We employ one reference-based metric USE [40] and one our newly proposed reference-free metric MRR to evaluate code summarization.

Code generation is a NL-PL task that generating a code snippet based on NL descriptions. We employ the Concode dataset [60] in Java where the input contains both NL texts and class contexts, and the output is a Java function. We evaluate it with BLEU-4, exact match (EM) accuracy, and CodeBLEU [61] that considers syntactic and semantic matches based on the code structure in addition to the n-gram match.

Code translation aims to migrate legacy software from one PL to another, where CodeXGLUE focus on translating functions from Java to CSharp and vice versa. Because we do not include C/Sharp in our updated version of pre-training dataset, we use the AVATAR [62], a parallel corpus for Java-Python program translation, for code translation task.

Code refinement is to detect which parts of code are buggy and fix them via generating a correct code sequence. We

employ two Java datasets provided by Tufano et al. [63] with various function lengths: small (fewer than 50 tokens) and medium (50-100 tokens). Due to the limit edit refining, there are large tokens overlap between source and target code. The EM measurement could better reflect the correctness of the refinement generation. We report EM and BLEU-4 scores for evaluation.

Clone detection aims to measure the similarity between two code snippets and predict whether they have the same functionality. We experiment with the Java data provided by Wang et al. [64]. We employ F1 score and accuracy for evaluating these two tasks respectively.

B. Results and Analysis

In this section, we value the re-pre-trained CodeT5 on downstream tasks. For better illustration, cgpt-CSN denotes the model was pre-trained with updated version of CodeSearchNet that we rebuilt using ChatGPT.

Code Summarization The initial two rows in Table IV reveal that the model pre-trained with cgpt-CSN outperforms the one trained with vanilla CSN on code summarization task, as indicated by higher USE and MRR scores across all six PLs. We further explore the effect of ChatGPT-generated comments data in fine-tuning phase, we rebuild the CSN code summarization datasets, denoted as cgpt-CSN-Sum. The results are shown in the middle two rows in Table IV. It should be noted that the ground truth for reference-based metrics USE on rebuilt CSN summarization test set are ChatGPT generated comments. We observe that model fine-tuned with ChatGPT-rebuilt summarization data outperforms the model fine-tuned with human-referenced data. These findings indicate that incorporating ChatGPT-generated data, both in pre-training and fine-tuning phases, advances code summarization performance.

The final two rows compare MRR scores between reference and ChatGPT-generated comments on the CSN summarization test set. ChatGPT-generated comments achieve markedly higher MRR scores across all six programming languages, aligning with findings from Section III. This consistency underscores the generalization capabilities across multiple programming languages.

Code Generation The vanilla input in Concode dataset contains both NL text and class environment context. To investigate the impact of ChatGPT-generated comments in fine-tuning stage, we also rebuild the Concode dataset by replacing the NL texts with ChatGPT-generated comments to form the updated inputs, as shown in Figure 5, and the updated version noted as cgpt-Concode. Table V shows that the model pre-trained with cgpt-CSN and fine-tuned with cgpt-Concode outperforms other training-tuning settings in three metrics. Compared to the model pre-trained on CSN and fine-tuned on Concode, ChatGPT-generated comments data in both training and fine-tuning phases contributes to 8.0%, 11.7% and 11.0% points improvement on EM, BLEU, and Code BLEU respectively. Particularly, we find that in

the fine-tuning phase, the ChatGPT enhanced cgpt-Concode contributes greater performance gains than pre-training.

We attribute this phenomenon to the characteristics of the NL-Code generation task itself. High-quality NL can better align the semantics with Code models, and the model learns better representations in the embedding space during the training process. In the NL-Code generation task, the quality of NL directly determines the semantic correlation between the input and the final target code. Therefore, in the fine-tuning stage, improving the NL quality of the fine-tuning data of the NL-Code generation task can significantly improve the quality of the generated code. These results demonstrate that high-quality comments data generated from ChatGPT could significantly advance the performance of the NL-code generation task.

Code Translation The experiment results of code translation task in the AVATAR testset are displayed in Table VI. We observe that the model pre-trained with cgpt-CSN data performs better than its counterpart pre-trained with human-commented CSN. Specifically, compared to the human-written comments, ChatGPT-generated comments in pre-training dataset achieve 3.7 and 1.67 CodeBLEU points higher performance in Java-to-Python and Python-to-Java translation, respectively. Somewhat surprisingly, code translation is an NL-unrelated task, and ChatGPT-generated NL in bimodal data can still improve the performance.

Code Refinement Experiment results of EM and CodeBLEU scores for code refinement tasks are shown in Table VII. The results indicate no significant performance differences between models trained with CSN and cgpt-CSN on both small and medium code refinement test sets. This implies that a deep comprehension of programming logic and syntax is essential for recognizing and correcting incorrect code patterns. The enhancement of NL comments in pre-training stage is less directly applicable to identifying and correcting code errors.

Clone Detection Table VIII displays the results of F1 scores, Precision and Recall on clone detection task. No obvious F1-scores difference is found between the two models that were pre-trained with CSN/cgpt-CSN, respectively, as the performance difference is no more than 0.5 percentage point. The inputs of the clone detection task are two code snippets; the similarity between them is measured to predict whether they have the same functionality. We speculate that the similarity between code snippets is less about their natural language descriptions and more about their code content. Therefore, the ChatGPT-generated high-quality comments data does not directly transfer to better performance in recognizing code clones, which may depend more on code-specific features.

Answer to RQ3 For natural language related code intelligence tasks, such as code summarization and NL-code generation, high-quality comments pre-training data rebuilt by ChatGPT could enhance their performance.

TABLE IV
EVALUATION RESULTS OF CODE SUMMARIZATION

pre-train	fine-tune	Javascript		PHP		Ruby		Java		Python		Go	
		USE	MRR	USE	MRR	USE	MRR	USE	MRR	USE	MRR	USE	MRR
CSN	CSN	0.5404	0.8102	0.6381	0.8102	0.5262	0.8189	0.6031	0.8083	0.5875	0.8294	0.6692	0.8580
cgpt-CSN	CSN	0.5420	0.8889	0.6429	0.8992	0.5375	0.8563	0.6111	0.9135	0.5911	0.9074	0.6763	0.9621
CSN	cgpt-CSN-Sum	0.7698	0.9623	0.8142	0.9202	0.7599	0.9477	0.8127	0.9342	0.6953	0.9529	0.8142	0.9120
cgpt-CSN	cgpt-CSN-Sum	0.7731	0.9665	0.8162	0.9617	0.7738	0.9617	0.8154	0.9553	0.6983	0.9552	0.8162	0.9244
human references		-	0.7710	-	0.8094	-	0.7988	-	0.8091	-	0.8153	-	0.8481
gpt-3.5-turbo		-	0.9580	-	0.9559	-	0.9694	-	0.9624	-	0.9089	-	0.9351

TABLE V
EVALUATION RESULTS OF NL-CODE GENERATION

pre-train	fine-tune	EM	BLEU	CodeBLEU
CSN	Concode	22.00	38.52	39.45
CSN	cgpt-Concode	29.60	48.31	49.25
cgpt-CSN	Concode	22.10	40.91	40.56
cgpt-CSN	cgpt-Concode	30.00	50.20	50.49

TABLE VI
EVALUATION RESULTS OF CODE TRANSLATION

pre-train	fine-tune	CodeBLEU	BLEU	EM
CSN	Java2Python	48.28	51.29	2.57
cgpt-CSN		51.98	56.24	2.31
CSN	Python2Java	55.52	56.82	1.21
cgpt-CSN		57.19	59.92	1.84

V. THREATS TO VALIDITY

Prompt Variability in ChatGPT. ChatGPT’s outputs can vary significantly with slight changes in the prompt structure, wording, or context. A notable threat to the validity arises from the prompt we used to indicate ChatGPT for generating code comments. We employed a simple designed prompt across different programming languages to relieve this threat. While this method ensures easy replication and consistency across different code snippets, it does not account for the potential variability and nuanced responses that might emerge from more elaborately designed prompts.

Evolving Nature of ChatGPT. A threat to the validity of our study stems from the versions of ChatGPT used in our research, specifically text-davinci-003 and gpt-3.5-turbo, which represent ChatGPT’s capabilities at a certain point in time. ChatGPT is continuously updated and improved by its

developers as a commercial product, leading to changes in its performance and output characteristics. While potentially beneficial, these updates can also introduce variability and unpredictability in the results. Therefore, our findings may have limited longevity.

Limitations of Evaluation Metrics. Another crucial threat to the validity of our study lies in the limitations of the evaluation metrics used for assessing code generation tasks. The metrics, like CodeBLEU, may not fully capture the quality and functionality of generated code in code intelligence tasks. However, these quantitative measures may overlook code correctness, potentially impacting the validity of our findings.

Limitations in Generalizability. The scope of our study’s conclusions is constrained by limitations in computing resources and the expenses related to using the OpenAI API. Consequently, our research focused solely on a specific pre-trained code model, and we did not extend our analysis to include open-source Code Large Language Models such as Code Llama [65].

VI. RELATED WORKS

Quality Issue of Dataset Recent advancements in learning-based methods have significantly propelled the field of code intelligence tasks. As a data-driven paradigm, the quality of code-comment paired data critically determines these methods’ effectiveness. Prior studies [21], [66]–[68] have investigated the inconsistencies between code and comment from different perspectives. Fluri et al. found that 3-10% of code comment changes lagged behind the corresponding code changes in seven Java open-source projects [69]. Such obsolete comments may provide misleading information to developers, leading them to write vulnerable code [67] and thus degrading the quality of the software. The noisy code comments data could degrade the performance of data-driven based learning models for code intelligence tasks [8].

The main focus of the research community is on developing customized models that can unleash the value of the available data in specific tasks. As mentioned in [70], [71], improving the quality of the training data is still a research opportunity for machine learning, including DL-based source code models. Sun et al. [8] proposed the first framework to improve the dataset quality for code search datasets. Their data cleaning framework, which consists of two subsequent filters: a rule-based syntactic filter and a model-based semantic filter, is considered to filter the noisy data. Xu et al. [72] investigated the data quality issue in the obsolete comment detection

TABLE VII
EVALUATION RESULTS OF CODE REFINEMENT

pre-train	fine-tune	EM	BLEU
CSN	Refine-small	21.41	77.41
cgpt-CSN		21.24	77.36
CSN	Refine-medium	13.90	89.39
cgpt-CSN		13.74	89.51

TABLE VIII
EVALUATION RESULTS OF CLONE DETECTION

pre-train	F1	P	R
CSN	0.9464	0.9455	0.9473
cgpt-CSN	0.9432	0.9358	0.9508

problem by proposing data cleaning and adversarial learning techniques. They found that the performance of DL models does improve with the cleaned training data. Unlike the above studies, our work tackles the data quality issue by rebuilding the dataset via LLMs, thereby replacing the noisy data with high-quality ones.

VII. CONCLUSION

We present a study comparing source code comments generated by ChatGPT, including GPT-3.5-turbo and text-davinci-003, to human reference comments of the corresponding codes. To provide a fair and comprehensive comparison, we propose two auxiliary tasks inconsistency detection and code search as two reference-free metrics to evaluate their comments quality of. We found that ChatGPT-generated comments are superior to human written ones. This finding has two implications. First, the research community may reconsider using human-written references as the gold standard for training and evaluating source code models. Second, it could be promising to use ChatGPT to generate high quality data for advancing downstream code intelligence tasks.

This exploration motivates the reconstruction of the widely used CodeSearchNet dataset, wherein we substituted all natural language comments with those generated by ChatGPT. Following this, we employed this rebuilt dataset to re-pre-train the CodeT5 model. Subsequently, the pre-trained CodeT5 was fine-tuned on 4 generation and 1 understanding tasks. The outcomes of our experiments were revealing; the CodeT5 model, when trained with ChatGPT-generated data, notably outperformed its counterpart trained with human-generated comments across several NL-related tasks, including code summarization, code generation, and code translation. These results demonstrate the practicality of using ChatGPT to enhance the pre-training dataset for advancing code intelligence tasks.

REFERENCES

- [1] "Program readability: Procedures versus comments," *IEEE Transactions on Software Engineering*, vol. 14, no. 9, pp. 1271–1279, 1988.
- [2] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, "The effect of modularization and comments on program comprehension," in *Proceedings of the 5th international conference on Software engineering*, pp. 215–223, 1981.
- [3] X. Song, H. Sun, X. Wang, and J. Yan, "A survey of automatic generation of source code comments: Algorithms and techniques," *IEEE Access*, vol. 7, pp. 111411–111428, 2019.
- [4] F. F. Xu, Z. Jiang, P. Yin, B. Vasilescu, and G. Neubig, "Incorporating external knowledge through pre-training for natural language to code generation," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 6045–6052, 2020.
- [5] M. Singh, J. Cambronero, S. Gulwani, V. Le, C. Negreanu, and G. Verbruggen, "Codefusion: A pre-trained diffusion model for code generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 11697–11708, 2023.
- [6] J. Shin, M. Wei, J. Wang, L. Shi, and S. Wang, "The good, the bad, and the missing: Neural code generation for machine learning tasks," *arXiv preprint arXiv:2305.09082*, 2023.
- [7] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proceedings of the 40th International Conference on Software Engineering*, pp. 933–944, 2018.
- [8] Z. Sun, L. Li, Y. Liu, X. Du, and L. Li, "On the importance of building high-quality training datasets for neural code search," in *Proceedings of the 44th International Conference on Software Engineering*, pp. 1609–1620, 2022.
- [9] Y. Xie, J. Lin, H. Dong, L. Zhang, and Z. Wu, "Survey of code search based on deep learning," *ACM Transactions on Software Engineering and Methodology*, 2023.
- [10] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, "Towards a big data curated benchmark of inter-project code clones," in *2014 IEEE International Conference on Software Maintenance and Evolution*, pp. 476–480, IEEE, 2014.
- [11] M. Lei, H. Li, J. Li, N. Aundhkar, and D.-K. Kim, "Deep learning application on code clone detection: A review of current knowledge," *Journal of Systems and Software*, vol. 184, p. 111141, 2022.
- [12] Y. Li, S. Wang, and T. N. Nguyen, "Dlfix: Context-based code transformation learning for automated program repair," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pp. 602–614, 2020.
- [13] C. L. Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.
- [14] T. Mamatha, B. R. S. Reddy, and C. S. Bindu, "A literature review on automated code repair," in *Proceedings of the 2nd International Conference on Recent Trends in Machine Learning, IoT, Smart Cities and Applications: ICMISC 2021*, pp. 249–260, Springer, 2022.
- [15] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.
- [16] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, et al., "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.
- [17] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 8696–8708, 2021.
- [18] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7212–7225, 2022.
- [19] D. Steidl, B. Hummel, and E. Juergens, "Quality analysis of source code comments," in *2013 21st international conference on program comprehension (icpc)*, pp. 83–92, Ieee, 2013.
- [20] A. T. Ying, J. L. Wright, and S. Abrams, "Source code that talks: an exploration of eclipse task comments and their implication to repository mining," *ACM SIGSOFT software engineering notes*, vol. 30, no. 4, pp. 1–5, 2005.
- [21] F. Wen, C. Nagy, G. Bavota, and M. Lanza, "A large-scale empirical study on code-comment inconsistencies," in *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pp. 53–64, IEEE, 2019.
- [22] L. Shi, F. Mu, X. Chen, S. Wang, J. Wang, Y. Yang, G. Li, X. Xia, and Q. Wang, "Are we building on the rock? on the importance of data preprocessing for code summarization," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 107–119, 2022.
- [23] X. Hu, G. Li, X. Xia, D. Lo, S. Lu, and Z. Jin, "Summarizing source code with transferred api knowledge," 27th International Joint Conference on Artificial Intelligence, IJCAI 2018, 27.
- [24] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.
- [25] Q. Guo, J. Cao, X. Xie, S. Liu, X. Li, B. Chen, and X. Peng, "Exploring the potential of chatgpt in automated code refinement: An empirical study," *arXiv preprint arXiv:2309.08221*, 2023.
- [26] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of ai programming assistants: Successes and challenges," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pp. 605–617, IEEE Computer Society, 2024.
- [27] M. Geng, S. Wang, D. Dong, H. Wang, G. Li, Z. Jin, X. Mao, and X. Liao, "Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning," 2024.
- [28] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

- [29] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, *et al.*, “Improving language understanding by generative pre-training,”
- [30] A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, “Learning and evaluating contextual embedding of source code,” in *International conference on machine learning*, pp. 5110–5121, PMLR, 2020.
- [31] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [33] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “Electra: Pre-training text encoders as discriminators rather than generators,” *arXiv preprint arXiv:2003.10555*, 2020.
- [34] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, “Intellicode compose: Code generation using transformer,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1433–1443, 2020.
- [35] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, *et al.*, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” *arXiv preprint arXiv:2102.04664*, 2021.
- [36] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Unified pre-training for program understanding and generation,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2655–2668, 2021.
- [37] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7871–7880, 2020.
- [38] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [39] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” *arXiv preprint arXiv:1904.09675*, 2019.
- [40] S. Haque, Z. Eberhart, A. Bansal, and C. McMillan, “Semantic similarity metrics for evaluating source code summarization,” in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, pp. 36–47, 2022.
- [41] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pp. 311–318, 2002.
- [42] E. Reiter, “A structured review of the validity of bleu,” *Computational Linguistics*, vol. 44, no. 3, pp. 393–401, 2018.
- [43] C. Van Der Lee, A. Gatt, E. Van Miltenburg, S. Wubben, and E. Krahmer, “Best practices for the human evaluation of automatically generated text,” in *Proceedings of the 12th International Conference on Natural Language Generation*, pp. 355–368, 2019.
- [44] J. Novikova, O. Dušek, A. C. Curry, and V. Rieser, “Why we need new evaluation metrics for nlg,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 2241–2252, 2017.
- [45] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text summarization branches out*, pp. 74–81, 2004.
- [46] S. Banerjee and A. Lavie, “Meteor: An automatic metric for mt evaluation with improved correlation with human judgments,” in *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, pp. 65–72, 2005.
- [47] J. Wieting, T. Berg-Kirkpatrick, K. Gimpel, and G. Neubig, “Beyond bleu: Training neural machine translation with semantic similarity,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4344–4355, 2019.
- [48] S. Stapleton, Y. Gambhir, A. LeClair, Z. Eberhart, W. Weimer, K. Leach, and Y. Huang, “A human study of comprehension and code summarization,” in *Proceedings of the 28th International Conference on Program Comprehension*, pp. 2–13, 2020.
- [49] J. Mahmud, F. Faisal, R. I. Arnob, A. Anastasopoulos, and K. Moran, “Code to comment translation: A comparative study on model effectiveness & errors,” *arXiv preprint arXiv:2106.08415*, 2021.
- [50] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- [51] D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. S. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, *et al.*, “Universal sentence encoder,” *arXiv preprint arXiv:1803.11175*, 2018.
- [52] S. Panthaplapackel, J. J. Li, M. Gligoric, and R. J. Mooney, “Deep just-in-time inconsistency detection between comments and source code,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 427–435, 2021.
- [53] T. Steiner and R. Zhang, “Code comment inconsistency detection with bert and longformer,” *arXiv preprint arXiv:2207.14444*, 2022.
- [54] F. Lv, H. Zhang, J.-g. Lou, S. Wang, D. Zhang, and J. Zhao, “Codehow: Effective code search based on api understanding and extended boolean model (e),” in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 260–270, IEEE, 2015.
- [55] X. Ye, R. Bunescu, and C. Liu, “Learning to rank relevant files for bug reports using domain knowledge,” in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, pp. 689–699, 2014.
- [56] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “A transformer-based approach for source code summarization,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4998–5007, 2020.
- [57] H. Wu, H. Zhao, and M. Zhang, “Code summarization with structure-induced transformer,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 1078–1090, 2021.
- [58] F. Mu, X. Chen, L. Shi, S. Wang, and Q. Wang, “Developer-intent driven code comment generation,” *arXiv preprint arXiv:2302.07055*, 2023.
- [59] D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. St. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, B. Strophe, and R. Kurzweil, “Universal sentence encoder for English,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (E. Blanco and W. Lu, eds.), (Brussels, Belgium), pp. 169–174, Association for Computational Linguistics, Nov. 2018.
- [60] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, “Mapping language to code in programmatic context,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 1643–1652, 2018.
- [61] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, “Codebleu: a method for automatic evaluation of code synthesis,” *arXiv preprint arXiv:2009.10297*, 2020.
- [62] W. U. Ahmad, M. G. R. Tushar, S. Chakraborty, and K.-W. Chang, “Avatar: A parallel corpus for java-python program translation,” *arXiv preprint arXiv:2108.11590*, 2021.
- [63] M. Tufano, C. Watson, G. Bavota, M. D. Penta, M. White, and D. Poshyvanyk, “An empirical study on learning bug-fixing patches in the wild via neural machine translation,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 28, no. 4, pp. 1–29, 2019.
- [64] W. Wang, G. Li, B. Ma, X. Xia, and Z. Jin, “Detecting code clones with graph neural network and flow-augmented abstract syntax tree,” in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 261–271, IEEE, 2020.
- [65] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [66] M. Linares-Vásquez, B. Li, C. Vendome, and D. Poshyvanyk, “How do developers document database usages in source code?(n),” in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 36–41, IEEE, 2015.
- [67] W. M. Ibrahim, N. Bettenburg, B. Adams, and A. E. Hassan, “On the relationship between comment update practices and software bugs,” *Journal of Systems and Software*, vol. 85, no. 10, pp. 2293–2304, 2012.
- [68] H. Malik, I. Chowdhury, H.-M. Tsou, Z. M. Jiang, and A. E. Hassan, “Understanding the rationale for updating a function’s comment,” in *2008 IEEE International Conference on Software Maintenance*, pp. 167–176, IEEE, 2008.

- [69] B. Fluri, M. Wursch, and H. C. Gall, “Do code and comments co-evolve? on the relation between source code and comment changes,” in *14th Working Conference on Reverse Engineering (WCRE 2007)*, pp. 70–79, IEEE, 2007.
- [70] Y. Zhao, L. Li, H. Wang, H. Cai, T. F. Bissyandé, J. Klein, and J. Grundy, “On the impact of sample duplication in machine-learning-based android malware detection,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–38, 2021.
- [71] C. Liu, X. Xia, D. Lo, Z. Liu, A. E. Hassan, and S. Li, “Simplifying deep-learning-based model for code search,” *arXiv preprint arXiv:2005.14373*, 2020.
- [72] S. Xu, Y. Yao, F. Xu, T. Gu, J. Xu, and X. Ma, “Data quality matters: A case study of obsolete comment detection,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 781–793, IEEE, 2023.

VIII. ACKNOWLEDGEMENT

For a special day, 2016-12-19 to 2023-12-19.